

HALP-Bench

Technical Analysis of a T4-Optimized
Supervised VLM Representation-Probing Pipeline

Code-grounded report for the supplied Jupyter notebook and exported Python source

Contents

Abstract	1
1 Introduction	1
2 Project Overview	2
3 Technical Foundations	2
4 Data and Input Processing	3
5 System Architecture and Code Organization	3
6 Detailed Methodology	4
7 Runtime bootstrap and dependency control	4
8 Dataset resolution and integrity	5
9 Reviewed-label acquisition and provenance	5
10 Grouped splitting and sampling	5
11 Model loading and processor interfaces	6
12 Architecture-aware representation extraction	6
13 SmolVLM2-specific extraction behavior	6
14 Qwen2.5-VL-specific extraction behavior	7
15 Persistent feature storage and resumability	7
16 Mathematical Formulation	8
16.1 Logistic link and linear baseline	8
16.2 Binary cross-entropy	8
16.3 Calibration metrics	8
16.4 Across-seed summaries and bootstrap uncertainty	9
17 Optimization and Learning Procedure	9

18 Implementation Details	10
19 Execution Workflow	10
20 Design Decisions and Rationale	11
21 Computational Considerations	11
22 Reproducibility and Reliability	12
23 Limitations and Technical Considerations	12
24 Overall Technical Assessment	13
25 Conclusion	13
26 Implementation Traceability	14
27 Configuration Snapshot	15
28 Source and Epistemic Boundaries	15
29 Final Quality-Control Statement	15

Abstract

This report provides a code-grounded technical analysis of the supplied HALP-Bench notebook and its exported Python source. The implementation is a research-oriented pre-generation probing pipeline for multimodal vision-language models (VLMs): it resolves an existing local HALP-Bench copy, validates benchmark and image integrity, acquires model-specific manually reviewed hallucination labels, constructs leakage-safe image-grouped splits, loads two T4-oriented VLM checkpoints, extracts architecture-aware internal representations, stores those representations in resumable HDF5 files, trains lightweight supervised probes, computes held-out metrics and bootstrap uncertainty summaries, performs error analysis, and validates the resulting artifact package.

The central methodological boundary is explicit in the implementation: hallucination labels are never inferred from the ground-truth answer, the benchmark question, or model-generated text. The VLM is evaluated before generation, and the downstream probe is treated as a predictor of the supplied model-specific reviewed annotation rather than as a system that intrinsically understands hallucination. The source also separates engineering observations, benchmark facts, and experimental artifacts, which reduces the risk of accidentally presenting an incomplete Colab run as a completed scientific result.

The analysis below emphasizes the relationship between code structure, mathematical formulation, numerical behavior, memory constraints, reproducibility, and scientific validity. It intentionally does not report experimental result values or reproduce notebook figures.

1 Introduction

The supplied project is best understood as a representation-probing study rather than a conventional end-to-end VLM benchmark. Its computational question is: can internal representations produced by a VLM, before answer generation, support prediction of a model-specific hallucination label that has been manually reviewed? The code operationalizes this question by pairing each image-question example with one or more internal feature vectors and then fitting lightweight binary classifiers on those vectors.

This design matters scientifically because it separates the representation being probed from the decision mechanism used to probe it. The VLM is not retrained. Instead, the VLM acts as a frozen feature generator, while logistic regression and a compact multilayer perceptron (MLP) provide controlled downstream readouts. The resulting experiment therefore targets the information content of the representation, subject to the quality and provenance of the reviewed labels.

The project also has a strong engineering dimension. It is explicitly optimized for Google Colab with an NVIDIA T4-class GPU, where transient memory, session resets, package-binary incompatibilities, and limited runtime can dominate practical reproducibility. The source therefore contains environment bootstrapping, memory guards, resumable checkpoints, image-manifest caching, per-sample failure logging, model-release logic, and a final artifact validation gate. These mechanisms are not peripheral conveniences; they form part of the experimental infrastructure that makes the intended study executable.

This report uses the exported Python source as the primary line-level traceability anchor and

treats the notebook as the conceptual execution container. The two supplied files describe the same project in notebook and exported-source form; the notebook has 55 cells, with 28 code cells and 27 Markdown cells, while the exported source contains roughly 22.6k lines and a large collection of helper functions supporting the same stages.

2 Project Overview

At the highest level, the pipeline is organized into the following computational stages: runtime preparation; persistent project-directory creation; narrow discovery of an existing HALP-Bench copy; dataset and image integrity auditing; acquisition and validation of model-specific reviewed labels; model registration; supervised table construction; grouped train/validation/test splitting; model API smoke testing; architecture-aware representation validation; persistent feature extraction; feature-file validation; probe training and validation-only configuration selection; final train-plus-validation refitting; one-time test evaluation; statistical summarization; error analysis; artifact writing; and a final validation and packaging gate.

The dependency structure is deliberately asymmetric. Feature extraction can proceed without supervised labels, because the labels are not needed to produce internal VLM representations. In contrast, supervised probe training is explicitly blocked unless real reviewed labels, a valid grouped split, and complete enough extracted features are available. This separation allows infrastructure and representation extraction to be validated independently from the supervised endpoint.

- Inputs: existing HALP-Bench metadata and images, VLM checkpoints/processors, and model-specific reviewed hallucination labels.
- Intermediate state: deterministic sample selection, grouped splits, model/runtime metadata, feature vectors, checkpoints, prediction files, and validation records.
- Outputs: HDF5 representation stores, probe result tables, predictions, uncertainty summaries, audit/validation documents, and a research artifact ZIP snapshot.

3 Technical Foundations

The scientific setup is binary supervised probing. For an image-question pair, the code assigns a binary target from the imported reviewed-label artifact and treats an extracted VLM representation as the explanatory variable. The probing model estimates the probability that the reviewed endpoint marks the example as hallucinated.

The VLM is not optimized against the probe loss. Its parameters remain frozen during extraction. This creates a clean separation between representation generation and supervised readout, but it also means that probe performance is evidence about linearly or nonlinearly accessible information in the frozen representation, not evidence that the original VLM internally implemented the same decision boundary.

The representation families have different semantic locations. VF is intended to represent a mean-pooled visual-encoder signal before multimodal fusion or projection, when the architecture exposes such a signal cleanly. VT is the hidden state at the final identifiable image-token

position in selected decoder layers. QT is the hidden state at the final non-padding query position in the same selected layers. The code is careful not to force this abstraction onto unsupported model wrappers.

4 Data and Input Processing

The dataset stage assumes an existing local HALP-Bench copy and, by design, does not download or replace the benchmark. The discovery code performs a bounded directory search under Google Drive, ranks candidate directories using explicit repository signals, and falls back to locating a ZIP archive when an extracted directory is absent. Extraction from an existing local ZIP is permitted, but the benchmark itself is not fetched from the network by this stage.

The benchmark schema audit expects question identifiers, image names, question text, ground-truth answers, category/source metadata, image-presence metadata, and the relational flag. The code then maps image basenames to actual local image paths through a cached manifest. Duplicate image basenames that resolve to different files are treated as an ambiguity rather than silently resolved; missing files block a full run.

The dataset fingerprint combines the benchmark CSV bytes with image names, sizes, and modification times. This fingerprint is inserted into the run signature so that persistent runs can be resumed only when the experimental configuration and data identity are consistent. The manifest itself is also keyed by benchmark path, size, and modification time, reducing repeated filesystem scans.

The supervised endpoint is intentionally model-specific. The source contains explicit URLs for the reviewed Qwen2.5-VL and SmolVLM label CSVs and stores them under a separate persistent directory. The loader prefers an explicit manually reviewed column and refuses to fall back to an automatic hallucination column when the manual field is missing. It computes disagreement between automatic and reviewed labels as a diagnostic, but never replaces the manual target with the automatic label.

Labels are normalized to a binary representation and joined to the benchmark on question identifier using a one-to-one validation contract. Label coverage must reach the configured 95 percent threshold. The code rejects duplicate identifiers on the label side and explicitly documents the semantics of the resulting target: one denotes hallucination according to the imported model-specific reviewed annotation, and zero denotes non-hallucination.

5 System Architecture and Code Organization

The software architecture is declarative at the model level and procedural at the experiment level. ModelConfig stores model identity, processor identity, dtype, device-map policy, layer-location fractions, generation defaults, random seeds, and output naming information. The active registry contains two configurations: HuggingFaceTB/SmolVLM2-2.2B-Instruct and Qwen/Qwen2.5-VL-3B-Instruct. The source deliberately avoids assuming that layer numbers or image-token identifiers are shared between the architectures.

The exported source contains 193 AST-visible function definitions and five classes, but those counts should not be interpreted as the number of conceptual algorithms. Many functions are

small adapters or validation helpers. The architecture is more naturally viewed as a set of subsystems: environment bootstrap; dataset/label resolution; VLM loading and smoke tests; representation adapters; persistent feature storage; probe training; statistical post-processing; artifact generation; and final validation.

- Runtime layer: package version auditing, fresh-process scientific-stack testing, optional binary repair, kernel restart coordination, CUDA diagnostics, and final dependency checks.
- Data layer: benchmark discovery, image indexing, provenance checks, label acquisition, schema normalization, grouped splitting, and dataset fingerprinting.
- Model/representation layer: processor adapters, hidden-state inspection, layer and token-position discovery, architecture-specific extraction, numerical retry logic, and feature validation.
- Supervised layer: scaling, logistic regression, MLP sweeps, validation-only selection, test-threshold locking, metrics, calibration, and bootstrap confidence intervals.
- Artifact layer: CSV/HDF5/JSON/Markdown outputs, manifest generation, audit report generation, validation of files on disk, and final ZIP packaging.

6 Detailed Methodology

The methodology proceeds in a fixed conceptual order, although the notebook contains defensive checks that allow some stages to be rerun independently. The following subsections describe what the code actually does and why each stage exists.

7 Runtime bootstrap and dependency control

The environment bootstrap intentionally imports only the Python standard library until the scientific stack has been tested. Package metadata is read without importing the binary libraries, which is important because importing NumPy, SciPy, or scikit-learn before a package repair can leave stale compiled objects in the current kernel. The notebook therefore tests the scientific stack in a fresh Python subprocess, checks declared version ranges with `packaging.specifiers`, and repairs the scientific wheel set only when the fresh-process test or declared constraints fail.

A clean restart is requested only when package files have actually changed. A small JSON state file under `/content` coordinates the restart so the post-restart pass can distinguish a normal initial pass from the verification pass. The process terminates without intentionally manufacturing a Python traceback, and the final import stage then verifies NumPy, SciPy, scikit-learn, Pandas, HDF5, PyTorch, Transformers, Accelerate, Datasets, and related dependencies.

The CUDA diagnostics then select a device, record total and free VRAM, inspect compute capability, and query BF16 support. The actual extraction logic is still centered on FP16 because the declared production profile uses float16. The code treats Accelerate placement limits as only a placement budget; it also checks real CUDA utilization during execution against a hard 13.5 GiB ceiling and a preferred 13.0 GiB peak.

8 Dataset resolution and integrity

The dataset discovery procedure is intentionally conservative. It walks only bounded directory depths, skips common cache/version-control folders, scores candidate roots, and requires the expected benchmark CSV before selecting a directory. The code never assumes that the dataset directory is literally named HALP-Bench, which makes it more robust to Drive organization changes.

Once a candidate is selected, the image index is built once and cached. The cache is invalidated when the dataset path, benchmark path, file size, or modification time changes. Each image basename is mapped to exactly one file. This is a practical integrity safeguard because a basename collision would otherwise make a question silently use the wrong image.

9 Reviewed-label acquisition and provenance

The supervised target is one of the most scientifically sensitive parts of the pipeline. The code makes the reviewed endpoint an explicit input rather than deriving a proxy label. It downloads only the two model-specific reviewed CSVs needed by the active registry, caches them on persistent storage, computes a SHA-256 digest, and creates a small bundle and manifest for later reuse.

There is a deliberate semantic firewall between manual and automatic labels. When the expected manual-review column exists, it is the only training target. If the manual column is absent but an automatic column is present, the code raises an error instead of silently accepting the automatic field. This is a strong anti-label-leakage and provenance-preservation choice because the two label constructions can encode different annotation behavior.

10 Grouped splitting and sampling

The experimental cohort is first sampled deterministically when MAX_SAMPLES is smaller than the available labeled frame. In the supplied configuration, the nominal full execution is capped at 8000 rows per model. The fixed sampling seed is 42, so the same model-specific dataframe and cap produce the same sampled cohort as long as upstream ordering and content remain unchanged.

The train/validation/test partition is grouped by image name rather than by individual question rows. This is a crucial leakage control because several questions may refer to the same image. GroupShuffleSplit creates an outer test split and an inner validation split. The code then tries up to 25 seed offsets and selects the valid partition whose train, validation, and test class prevalences are jointly closest to the overall labeled prevalence while also requiring both classes in each partition.

After selection, the code asserts pairwise disjoint image groups and separately validates question-identifier disjointness. The test set is therefore isolated at both the group and identifier levels. The source correctly avoids claiming that this procedure reproduces any hidden official split; it is an internally constructed research split whose properties are explicit in the run artifacts.

11 Model loading and processor interfaces

The loader uses current Hugging Face multimodal APIs. SmolVLM2 is loaded through an image-text-to-text auto class, while Qwen2.5-VL first attempts a generic multimodal auto class and falls back to the architecture-specific Qwen class if the generic API is unavailable. Models are placed using an automatic device map and loaded in float16.

Input construction is architecture-aware. Qwen uses its chat template and qwen-vl-utils to construct image/video inputs, whereas SmolVLM2 uses the processor chat-template route with a local image path. Generation is never invoked in the representation experiment: inputs are constructed with `add_generation_prompt=False` and inference calls request hidden states or selected hooks rather than sampled text.

A one-sample smoke test is performed before the expensive extraction run. The smoke path verifies image preprocessing, a single forward pass, hidden-state availability and shapes, model-device behavior, and identifiable image/query token positions. This is an important boundary because it moves wrapper/API failures to a cheap early stage rather than allowing them to fail deep inside a long extraction run.

12 Architecture-aware representation extraction

The representation subsystem has three families. VF is a visual-only vector obtained by mean pooling a validated visual-encoder signal. VT and QT are decoder hidden-state vectors sampled at carefully identified sequence positions. The implementation validates tensor rank, batch size, sequence bounds, finiteness, and feature dimensionality before writing to persistent storage.

Decoder layers are selected according to the HALP-style normalized locations 0, 0.25, 0.50, 0.75, and 1.0, but the actual decoder indices are constructed from the runtime depth. The helper maps the early location to layer 1 and the later locations to $\text{floor}(\text{fraction} \times L)$, clipped to the valid range 1 through L. Thus the method is architecture-aware without hard-coding a single depth.

The image-token position is discovered from the model configuration and the actual tokenized input. For VT, the final identifiable image-token position is used. For QT, the final non-padding position from the attention mask is used when available, with a tokenized-length fallback. This is a meaningful implementation detail: the representation is tied to the sequence produced by the current processor rather than an assumed fixed token offset.

13 SmolVLM2-specific extraction behavior

The SmolVLM2 path preserves a model-compatible 384-pixel preprocessing edge because the checkpoint configuration is coupled to its expected visual tokenization. The source explicitly notes that an earlier 336-pixel override created a mismatch between actual visual patch tokens and the processor-inserted image-token count. The corrected path disables image splitting and performs a single 384-pixel image transformation per sample.

SmolVLM2 normally exposes hidden states directly from the multimodal forward. The

extraction code can also reuse cached image hidden states for repeated image paths when the optimization switch is enabled. That optimization removes redundant visual computation without changing the conceptual VF/VT/QT targets; the cache is keyed by image path and reused only when the same image is encountered again.

14 Qwen2.5-VL-specific extraction behavior

Qwen2.5-VL receives a stricter T4-oriented path because its visual preprocessing rounds dimensions to a 28-pixel grid. The code begins from a target of 256 visual image tokens and enforces an upper patch budget four times that value. If the processor rounds a sample upward, the maximum pixel budget is reduced adaptively by multiplying it by 0.88 until the resulting grid falls within the declared limits or the minimum pixel budget is reached.

The Qwen extraction avoids retaining the entire hidden-state tuple. Instead, it registers forward hooks only on the selected decoder blocks and copies the requested VT and QT vectors to CPU. The visual encoder output is also captured by a hook during the same multimodal forward and mean-pooled to form VF. This reuse of the same forward pass is particularly important on a 16 GiB-class GPU because it removes an additional vision-only pass and limits transient activation residency.

The forward path includes numerical retries. If the fast attention configuration produces a NaN/Inf-related runtime failure, the code can retry with math SDPA and then a safer eager-attention path. The retry modes are explicit, bounded, and recorded. A genuine non-finite model output is not repaired by clipping or zero-filling; the sample is instead marked failed if all allowed recovery paths are exhausted.

15 Persistent feature storage and resumability

Each model receives one HDF5 representation file. Metadata arrays store question identifiers, image names, labels where available, status, category/source text, and question text. Feature datasets are added lazily as representation keys become available, and each feature matrix uses one row per selected example with a float16 storage dtype, gzip compression, and a chunk shape optimized for one-row writes.

A JSON checkpoint tracks completed identifiers, failed identifiers, model identity, run signature, extraction settings, progress, numerical retry counts, and timestamps. On resume, completed IDs are read from both the checkpoint and the HDF5 status array, and already-complete samples are skipped. This means a Colab interruption does not require a complete recomputation of the expensive VLM stage.

The main loop processes one sample at a time, periodically flushes the HDF5 file, periodically writes the checkpoint, records telemetry, frees Python and CUDA caches at defined intervals, and prioritizes previously failed samples on resume. These choices favor fault containment and resumability over maximum throughput.

16 Mathematical Formulation

The notebook provides an explicit mathematical statement for the probe stage. Let $h(x)$ in $\mathbb{R}^d$ be an extracted representation for example x , and let y in $\{0,1\}$ be the model-specific reviewed hallucination label. The probe estimates a conditional probability through a logistic link.

16.1 Logistic link and linear baseline

The probability model is

$$p_{\theta}(y = 1 \mid h) = \sigma(f_{\theta}(h)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1)$$

For logistic regression, the scoring function is linear in the representation. Each feature contributes through a learned weight, while the intercept absorbs the global class offset. Because the VLM itself is frozen, the linear probe can be interpreted as a transparent readout of the information present in the selected representation.

For the linear baseline,

$$f_{\theta}(h) = w^{\top} h + b, \quad (2)$$

where $w \in \mathbb{R}^d$ is the coefficient vector and $b \in \mathbb{R}$ is the intercept.

16.2 Binary cross-entropy

The MLP is trained with binary cross-entropy via `BCEWithLogitsLoss`. The implementation therefore optimizes logits directly and applies the sigmoid only when probability values are needed for evaluation. This is numerically preferable to computing $\log(\text{sigmoid}(\text{logit}))$ by hand because the framework loss can use stable internal transformations.

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (3)$$

16.3 Calibration metrics

The code reports Brier score as a probability-calibration-sensitive loss and computes expected calibration error with ten fixed bins over the unit interval. For each occupied bin, the empirical positive rate is compared with the mean predicted probability, weighted by the fraction of test examples in that bin.

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2. \quad (4)$$

For fixed confidence bins S_b , expected calibration error is computed as

$$\text{ECE} = \sum_{b=1}^B \frac{|S_b|}{N} |\text{acc}(S_b) - \text{conf}(S_b)|. \quad (5)$$

16.4 Across-seed summaries and bootstrap uncertainty

For each model, representation, and probe family, the final summary aggregates results across the configured seeds using the arithmetic mean and sample standard deviation. The bootstrap routine then resamples held-out test examples with replacement 1000 times and forms a percentile 95 percent interval for AUROC, discarding degenerate resamples that contain only one class. The source correctly states that this interval captures sampling variability in the held-out examples; it is not uncertainty across independent VLM checkpoints.

$$\bar{s} = \frac{1}{K} \sum_{k=1}^K s_k, \quad s_{\text{std}} = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (s_k - \bar{s})^2}. \quad (6)$$

17 Optimization and Learning Procedure

The probe stage uses two model families. Logistic regression is the linear reference model. The MLP has the actual extracted feature dimension as its input size, followed by three hidden layers, ReLU activations, BatchNorm1d, optional dropout, and a single scalar output. Three compact candidate configurations are evaluated per representation and seed. Their hidden widths are 128-64-32, 256-128-64, and 512-256-128, with progressively stronger regularization and different learning rates.

MLP optimization uses AdamW. The class-weighting rule compares the larger class count with the smaller class count; when the ratio reaches 1.5, weighted training is enabled. For the MLP this becomes BCEWithLogitsLoss with a positive-class weight equal to negative-count divided by positive-count. Logistic regression uses scikit-learn `class_weight="balanced"` under the same trigger.

Early stopping is validation-driven. After each training epoch, validation probabilities are computed under inference mode and validation AUROC is recorded. The best model state is copied to CPU memory, and training stops when validation AUROC fails to improve for the configured patience window. This preserves the validation boundary and avoids peeking at the test set during optimization.

For the final evaluation, the selected configuration is refit on the union of train and validation data only. Crucially, normalization statistics are recomputed from this union rather than from the earlier train-only scaler. The test threshold is the threshold selected on validation using maximum F1, then balanced accuracy, and finally proximity to 0.5 as tie-breakers. The locked threshold is reused for final test metrics.

A specific BatchNorm failure mode is handled in both training and refitting: when the dataset size would otherwise create a singleton final batch, the requested batch size is reduced by one. This preserves every sample while preventing BatchNorm1d from receiving a batch of size one.

18 Implementation Details

The extraction and supervised stages intentionally use different hardware policies. VLM inference is the dominant CUDA workload, so probe training is configured for CPU execution. The code limits CPU threads to a small number and disables pinned host memory by default because probe tensors are small and do not need a persistent host-memory pipeline.

The source uses `inference_mode` during feature extraction and model evaluation, `autocast` with `float16` on CUDA, explicit cache cleanup, and model release between active models. It also requests `logits_to_keep=1` where supported, which reduces unnecessary output materialization in modern Transformers implementations.

Feature conversion is deliberately two-stage. The raw model tensor is first checked for finiteness in `float32` before conversion. Only then is it cast to `float16` and copied to CPU. The post-cast vector is checked again so an overflow caused by the storage dtype is distinguished from a genuinely non-finite model output. This distinction improves debugging and prevents silent numerical corruption.

The code separates supported and unsupported representation families. A representation is only written when an adapter can verify the corresponding model signal. For example, the current Qwen implementation captures raw visual output during the multimodal forward, while a generic fallback does not fabricate VF simply because a similarly named tensor exists. Unsupported quantities are recorded and the validation stage reports what is actually present.

19 Execution Workflow

A conceptually complete run starts by selecting the production configuration: full mode, an 8000-row deterministic cap per model, three probe seeds, two active VLMs, a 256-image-token Qwen target, and a 13.5 GiB hard CUDA ceiling. The environment bootstrap then validates or repairs the runtime, the Drive structure is mounted, and the existing dataset is resolved.

The benchmark is audited before any long model call. The reviewed labels are acquired and validated. Each active model receives its own labeled dataframe and grouped partition. A model smoke test then confirms that its current processor and wrapper can execute one forward pass and expose the intended representation interfaces.

Feature extraction writes one HDF5 file per model and is resumable. After extraction, the validation stage inspects dimensions, row counts, status values, feature dtypes, numerical finiteness on completed rows, and feature-key presence. Only a validated feature file proceeds into the probe stage.

The probe stage loads matrices in HDF5 order and then restores the caller-defined row order. Each representation/seed receives a logistic baseline and an MLP configuration sweep. Once validation selects the MLP configuration and threshold, the final model is refit on train plus validation, evaluated once on test, and its predictions are saved for independent rescoring.

Post-processing then produces seed summaries, bootstrap AUROC intervals, error-analysis tables, research figures in the original project, manifest metadata, an audit report, and a final disk-level validation gate. This report does not reproduce those figures because the requested document is intentionally figure-free.

20 Design Decisions and Rationale

Several design choices are especially strong because they connect scientific validity with implementation mechanics.

First, the reviewed-label boundary is explicit. The pipeline refuses to turn `gt_answer` or question content into a proxy target. This prevents the supervised problem from quietly becoming a different task than the one defined by the supplied annotation files.

Second, image-grouped splitting is treated as part of the scientific method rather than a preprocessing convenience. When multiple questions share the same image, a row-wise split could place visually identical evidence in both training and test partitions. `GroupShuffleSplit` directly addresses that contamination channel.

Third, the source distinguishes a placement memory budget from a measured device budget. `Accelerate` can constrain parameter placement, but transient activations and attention work can still exceed the practical memory envelope. The additional real-CUDA check is therefore a meaningful systems safeguard.

Fourth, Qwen extraction is implemented with selected-layer hooks instead of retaining every hidden-state tensor. That is a direct systems response to the memory cost of long multimodal sequences and large hidden-state tuples. It also keeps the extracted feature definition explicit: only the requested VT and QT vectors are copied.

Fifth, the final refit recomputes normalization on train plus validation only and releases the validation-selected MLP before allocating the refit model. This simultaneously fixes a methodological issue in the scaling boundary and a practical memory issue in the Colab runtime.

21 Computational Considerations

Dataset indexing is approximately linear in the number of files scanned because the image manifest walks the selected dataset tree once. The cached manifest reduces repeated scans. Duplicate-basename validation and file-stat fingerprinting are also linear in the number of indexed image files.

For each active model, the expensive extraction stage scales approximately with the number of selected samples times the cost of one multimodal forward pass. Because the production path uses batch size one, the dominant memory term is activation size for a single example rather than a large sample batch. The Qwen hook strategy reduces the amount of hidden-state data retained after the forward.

HDF5 storage is row-oriented at the feature dataset level, with chunk shape one by feature dimension. This is appropriate for incremental single-sample writes, although it is not necessarily optimal for very large contiguous matrix reads. The probe reader compensates for HDF5 fancy-indexing behavior by reading samples in sorted HDF5 order and restoring the original split order afterward.

The MLP training cost is comparatively small because the probe runs on already-extracted features. Its computational cost is roughly proportional to the number of training examples,

the number of epochs actually executed, and the total parameter count of the chosen network. The three-configuration sweep multiplies this cost by the number of candidate configurations and seeds, but it remains small relative to VLM feature extraction.

The bootstrap stage costs approximately $O(BN)$ for $B=1000$ bootstrap replicates and N test examples, with AUROC computed only on non-degenerate resamples. This cost is acceptable relative to VLM extraction and has the advantage of using only saved predictions, so it does not consume additional VLM memory.

22 Reproducibility and Reliability

Reproducibility is addressed at several layers. The experiment cap, sampling seed, probe seeds, model registry, dataset fingerprint, optimization profile, Qwen pixel budgets, SmolVLM2 preprocessing, checkpoint interval, and feature dtype all contribute to the run signature. A persistent run is resumed only when its stored signature matches the current configuration and the previous run was not already marked complete.

Randomness is explicitly controlled through Python, NumPy, and PyTorch seeds. The code exposes a `STRICT_REPRO` switch that enables deterministic algorithms and deterministic cuDNN settings when requested. The default is not strict determinism, which is reasonable for an interactive Colab workflow but means bitwise identity should not be assumed across all execution paths.

Reliability is further improved through multiple validation boundaries: a fresh-process scientific-stack test, current-kernel functional tests, dependency checks, dataset schema checks, label-join checks, image preflight, model smoke tests, representation validation, HDF5 alignment checks, prediction-file validation, artifact-file checks, and a final hard failure gate. The archive itself records that it is an artifact snapshot rather than a self-contained dataset-and-model release.

23 Limitations and Technical Considerations

The most important limitation is that the supervised target is inherited from the external reviewed-label files. The code is careful about provenance, but probe validity still depends on how those reviewed annotations were created, how completely they cover the benchmark, and whether the annotation semantics are stable across model families. The supplied code does not itself establish annotator agreement or label reliability beyond the consistency checks it performs.

The grouped split controls image-level leakage, but it does not remove all possible dataset dependencies. Similar question wording, category composition, or source-specific structure can still correlate across partitions. The code includes category/source metadata for later error analysis, but the current split optimization is defined primarily around image groups and class prevalence.

The 8000-row cap is an explicit computational compromise, not a claim of full 10k-row benchmark coverage. The notebook labels the mode as a capped deterministic subset and keeps the cap visible in the run signature. Any comparison against a full 10k-row benchmark should therefore account for this sampling design.

The MLP architecture contains BatchNorm1d and optional dropout, so small training sets can introduce additional estimation noise in batch statistics. The singleton-batch guard prevents a hard runtime failure but does not eliminate all small-sample effects. Increasing the sample count or using a normalization method designed for very small batches would change the modeling assumptions and should be treated as a methodological variation rather than a silent implementation tweak.

The class-weighting trigger uses a single imbalance ratio threshold. When the ratio is crossed, logistic regression uses balanced class weights and the MLP uses a positive-class weight derived from the class counts. If the positive class is actually the majority, that positive weight can be below one. The behavior is intentional and mechanically consistent with the formula in the code, but the scientific interpretation of weighting should be checked against the observed class direction for each run.

The bootstrap interval is conditional on the saved test predictions and a single fitted model per seed. It does not propagate uncertainty from model training, label review, or alternative checkpoints. The notebook correctly limits the interpretation to held-out example sampling, but a broader uncertainty analysis would require additional experimental structure.

Finally, the implementation is strongly coupled to current library APIs, model wrapper structures, and Colab behavior. The architecture adapters are designed to fail explicitly when expected tensors are not exposed, which is preferable to silent corruption, but it means future Transformer changes may require adapter maintenance.

24 Overall Technical Assessment

The implementation is technically coherent and unusually attentive to the boundary between experimental validity and engineering reliability. Its strongest aspect is not any single model component but the way the pipeline enforces provenance, leakage control, numerical checks, persistent state, and test-set isolation across the whole workflow.

From a research-software perspective, the code demonstrates competence in multimodal Transformer interfacing, feature extraction, HDF5 persistence, PyTorch and scikit-learn integration, runtime diagnosis, memory-aware execution, and statistical post-processing. The use of architecture-specific adapters instead of assuming a universal VLM interface is particularly appropriate for a comparative representation study.

The main remaining scientific dependencies are external to the feature-extraction machinery: the validity of the reviewed hallucination labels, the representativeness of the capped cohort, and the scope of inference that can be drawn from a small set of frozen checkpoints and a small set of random seeds. The source already documents these boundaries well and, importantly, refuses to convert incomplete runs into fabricated scientific conclusions.

25 Conclusion

The supplied HALP-Bench project implements a pre-generation supervised probing framework for multimodal VLM representations. It begins with a validated local benchmark, imports model-specific reviewed hallucination annotations, constructs image-grouped leakage-safe

partitions, and extracts VF, VT, and QT representations using architecture-aware logic. These representations are persisted in resumable HDF5 stores, validated for alignment and numerical integrity, and passed to transparent logistic and compact MLP probes with validation-only model selection and threshold tuning.

The code is equally concerned with scientific correctness and practical execution. T4-specific preprocessing limits, selected-layer Qwen hooks, shared-forward visual capture, numerical retry policies, checkpointed extraction, CPU-resident probes, and hard artifact validation all respond to concrete failure modes. The resulting project therefore communicates more than a working model pipeline: it demonstrates an understanding of how data provenance, leakage control, representation semantics, numerical stability, reproducibility, and systems constraints interact in a real research workflow.

26 Implementation Traceability

The exported Python source is used as the primary line-level traceability anchor. The ranges below identify the major implementation blocks; they are intentionally broad enough to remain useful when navigating the generated source in a code editor.

Subsystem	Source range	Technical role
Runtime configuration and environment policy	Lines 41-152; 217-1506	GPU budget, dependencies, reproducibility and safety policy
Dataset discovery and integrity audit	Lines 1535-1997	Existing-data resolution, schema validation, image manifest, fingerprint
Reviewed-label acquisition	Lines 2000-2860	Model-specific reviewed target, coverage, provenance and label semantics
Model registry and grouped split	Lines 2820-3060	Declarative model definitions and image-grouped train/validation/test split
Representation adapters and validation	Lines 6800-8960	Smoke validation, layer selection, token positions, VF/VT/QT semantics
Persistent extraction and HDF5 storage	Lines 8980-12680	Run signature, checkpoints, extraction loop, retries, telemetry
Feature-file validation	Lines 12900-14320	Alignment, numerical checks, feature dimensions and completion status
Probe architecture and training	Lines 14250-17650	Logistic baseline, MLP, scaling, selection, refitting, test evaluation
Statistics and bootstrap uncertainty	Lines 17650-19590	Seed summaries, bootstrap AUROC intervals and post-hoc analysis
Error analysis and artifacts	Lines 19590-22664	Prediction analysis, figures, manifest, audit, final validation and archive

27 Configuration Snapshot

Setting	Configured value
Execution mode	full, capped at 8000 rows per active model
Active VLMs	SmolVLM2-2.2B-Instruct; Qwen2.5-VL-3B-Instruct
Probe seeds	42, 52, 62
Feature storage	float16 HDF5 with gzip compression
Qwen visual budget	256 target image tokens; 128-token minimum retry floor
CUDA safety	13.5 GiB hard ceiling; 13.0 GiB preferred peak
Split grouping	image name
Label coverage threshold	95 percent
Bootstrap	1000 percentile resamples; 95 percent interval
Probe device	CPU

28 Source and Epistemic Boundaries

This report describes implementation behavior, not unobserved intent. Where the code contains an explicit scientific contract, that contract is reported directly. Where a rationale is inferred from the implementation, the wording is intentionally conservative. No experimental performance values, benchmark conclusions, or new empirical claims are introduced here. The report also does not assert that any prior notebook output represents a completed final study.

29 Final Quality-Control Statement

The deliverables accompanying this report were generated from the supplied project source. The LaTeX version is authoritative; the PDF is compiled from that source; and the Word version mirrors the same report structure and mathematical content as closely as the format permits. The report is deliberately date-free and contains no figures, screenshots, plots, result visualizations, or fabricated experimental outcomes.